Trust-Aware Continual Learning for IoT Intrusion Detection

Authorizing drift-triggered model updates with traffic evidence, an out-of-band reputation signal, and quarantined, reversible updates

Project report (report_ids)
VIT Project-I, Review 2 proof-of-concept

Repository: github.com/akarshdixit/trust-ids-poc

Contents: 0 Review-panel concerns - 1 Problem and goal - 2 System structure - 3 Repository layout - 4 Datasets - 5 Data processing and features - 6 Algorithms and training - 7 Experimental setup - 8 Results - 9 Comparison with other models - 10 Live experiment - 11 Robustness and the quarantine extension - 12 Related work and novelty - 13 Software, packages and backend - 14 Visualizations - 15 Testing - 16 How to run - 17 Limitations and future work - References

0. Review-panel concerns and how they are addressed

The review panel raised six shortcomings. Everything below was added on top of the existing project: the original pipeline, its parameters and its committed results are unchanged (tests check that the original arms reproduce their committed logs exactly).

Panel concern	What was added	Where	Headline result
1. No comparison with other models; which is better / more efficient?	Benchmark of 16 detectors (8 supervised batch, 2 unsupervised, 5 online, plus ours) on the same temporal split; every defence re-run on 5 different base learners	Sections 9.1-9.3; experiments/benchmark.py, sweeps.py models	Clean detection: best F1 1.0000 (N-BaIoT), 0.9926 (IoT-23); ours 0.9954 / 0.9898 at 24 / 5 us per record. Under poisoning the full gate holds 1.00 in 8 of 8 learner/dataset pairs whose detector works; learning everything is poisoned in 7 of those 8.
2. How was the dataset processed?	Step-by-step pipeline with measured effect of each step, splits, sampling and stream construction	Section 5.1-5.3; experiments/data_profile.py	Raw N-BaIoT features span 18.5 orders of magnitude; after signed log + z-score, 0. No missing or infinite values in any file.
3. What are the features, how are they defined and used?	Catalogue of all 115 N-BaIoT and 21 IoT-23 features with meanings, importance analysis by aggregation and time window	Section 5.4-5.5; data/features.py	Long time windows (~10 s, ~1 min) of per-host packet counts and sizes carry most signal on N-BaIoT; protocol and packet counts on IoT-23.
4. How were the models trained?	Training procedure and every hyperparameter, for the scenario pipeline and for each benchmark model	Section 6.7	400 labelled calibration rows, then online SGD only on authorized updates; benchmark models trained on 70% earliest data.
5. Accuracy is too low	Per-phase accuracy breakdown, clean held-out accuracy, live accuracy	Sections 8.3, 9.1, 10	The 0.55 is an average over phases where the attack is deliberately disguised as benign. On clean held-out traffic ours scores 0.995; live on a real capture, 0.997.
6. No live experiment	Real-time replay of a real 24-hour capture in its original order and timing, a live dashboard, and an ingest API for raw Zeek logs	Section 10; experiments/live_experiment.py, live_feed.py, backend /live	21,140 live flows: gates 0.997, learning everything 0.134. Capacity 13,766 flows/s for all six systems.

An honest note on 'our model works best'. Two different things are compared. As a plain detector on clean data, our online logistic regression is *not* the most accurate model: several batch models reach F1 1.000 on N-BaIoT, ours 0.995. It is the fastest and among the smallest, and it is trained on only 400 rows. The project's contribution is not the classifier but the authorization gate around it, and there the comparison is clear: whatever learner is inside, the reputation-aware gates are the only defences that keep it from being poisoned, in the scenarios and on live traffic.

1. Problem and goal

An IoT intrusion detection system (IDS) deployed on a changing network has to keep learning: devices are added, firmware changes and benign traffic drifts. Continual learning makes that possible, but it also opens an attack surface. A patient attacker can shape traffic so that it drifts slowly from attack towards benign; if the IDS adapts to every detected drift, it gradually learns that the attack is normal (*slow poisoning*, the 'boiling frog' attack).

The goal of this project is an **authorization gate** that sits between drift detection and the model update. Every proposed update must be authorized. The gate combines two kinds of evidence:

- **Traffic evidence** computed from the traffic window itself (attribution consistency, topology diversity, temporal stability). It is cheap but attacker-influenceable.
- **An out-of-band reputation signal** (where the traffic goes: destination reputation or C2 cost), which is independent of the traffic features and therefore not controlled by the attacker.

This version adds a **quarantine extension**: the reputation channel is treated as sequential, fallible evidence about a whole drift episode; updates are applied provisionally and rolled back when the evidence turns against them. The research question is whether such a gate keeps the benefits of continual learning on benign drift while refusing poisoning, including when the reputation feed is noisy or late.

Honest scope. No public IoT dataset carries a reputation field, so the signal is simulated from ground-truth origin (benign 1.0, attack 0.05) and then degraded on purpose. The claim is what an authorization mechanism achieves *once it has an independent channel*, not that the gate detects attackers by itself.

2. System structure

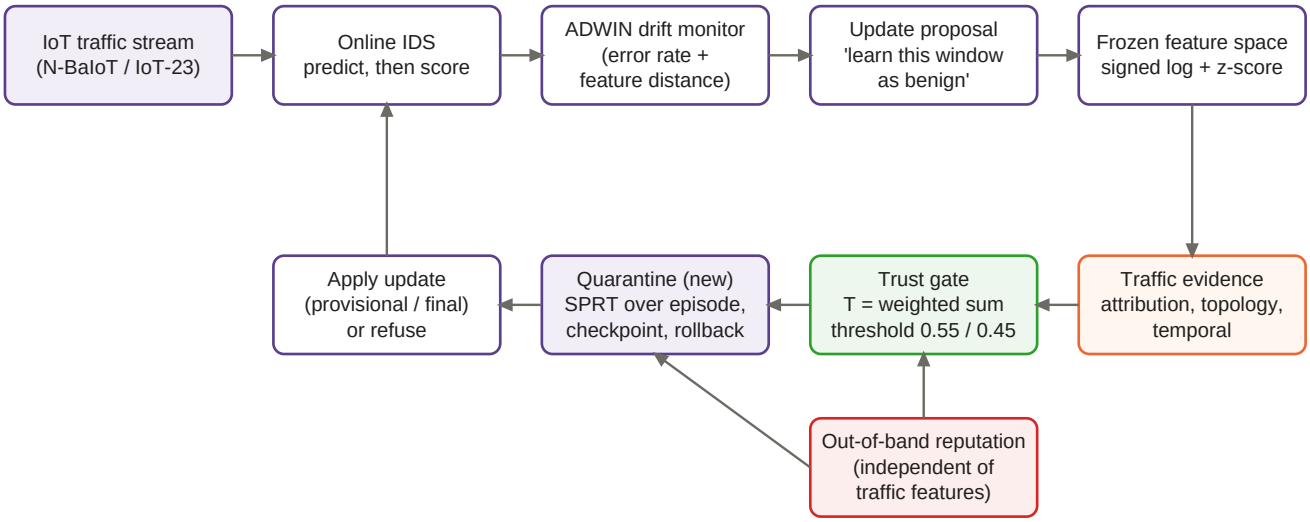


Figure 1. Pipeline. Every arm sees the same stream and the same drift trigger; the arms differ only in how they decide whether to apply a proposed update.

Per 20-record round:

- **Prequential evaluation.** Every arm predicts each record first and is scored afterwards (test-then-train), so no arm ever sees a label before predicting it.
- **Drift trigger.** A shared ADWIN detector watches the frozen static arm's error rate (optionally also each record's distance from the calibration centre). A change point opens a 15-round drift-active

window.

- **Proposal.** Only in drift-active rounds is an update proposed: 'learn this window as benign'. Outside it nobody learns.
- **Decision.** Each arm's policy (never, always, gate, EWC, quarantine) decides; authorized updates are applied with River's learn_one.

Six arms run side by side, which makes every comparison a controlled experiment:

Arm	Policy
Static (never updates)	Trained on calibration data once, never updates. Reference for 'no adaptation'.
Ungated (learns everything)	Applies every drift-triggered proposal.
Traffic-only gate	Traffic evidence only, no reputation veto (stands in for ExpIDS-style drift gating).
Full gate (one-shot veto)	Traffic evidence plus a one-shot reputation veto (the Review-2 mechanism).
EWC (lambda = 50)	Applies every proposal, with Elastic Weight Consolidation anchoring weights to calibration. Tests whether the gate's benefit is merely 'not forgetting'.
Quarantine gate (new)	Sequential reputation test over the drift episode, provisional updates, rollback (section 6.6).

3. Repository layout

Path	Role
models/online_ids.py	OnlineIDS (StandardScaler | LogisticRegression, ADWIN) and EWCOnlineIDS
evidence/featurespace.py	Frozen signed-log + z-score space (trust boundary)
evidence/extract.py	Attribution, topology and temporal evidence
gate/trust_gate.py	One-shot gate: weighted trust score + reputation veto
gate/quarantine.py	SPRT, provisional updates, checkpoints, rollback (new)
data/load_nbaiot.py	N-BaIoT loader and the two scenario streams
data/load_iot23.py	IoT-23 Zeek conn-log parser and streams (new)
data/synth_stream.py	Synthetic 20-feature streams (regression check)
data/reputation.py	Imperfect reputation feed: noise, flips, lag (new)
data/fake_nbaiot.py	Fake N-BaIoT-format files for tests (new)
experiments/run_scenarios.py	Orchestrator; writes results/*_log_<tag>.json
experiments/streams.py	build_stream(): one stream builder for CLI, API, app
experiments/sweeps.py	Seeds, reputation, EWC and quarantine sweeps (new)
experiments/make_plots.py	Static figures per data source
backend/server.py, console.html	FastAPI backend, SSE logs, live console (new)
app.py, charts.py, equations.py	Streamlit demo, Altair charts, equations
tests/	108 pytest tests
results/	Logs (.json) and figures (.png)
data/features.py	Meaning of every N-BaIoT and IoT-23 feature (new)
experiments/benchmark.py	16-model detector benchmark (new)
experiments/data_profile.py	Data quality, processing effect, importance (new)
experiments/live_experiment.py	Real-time replay of a real capture (new)
experiments/live_feed.py	Streams a live Zeek conn.log to the backend (new)
dashboard.py, backend/live.html	Comparison dashboards, live dashboard (new)

Path	Role
PROJECT_GUIDE.txt	Changes, how to run, meaning of every chart

4. Datasets

4.1 N-BaIoT (primary)

UCI dataset 442 (Meidan et al., 2018): traffic of nine commercial IoT devices infected with Mirai and BASHLITE (Gafgyt). Each row is a packet window summarised into 115 statistical features (packet size, jitter, counts, covariance and correlation over 5 time windows, for host, channel and socket aggregations), so no packet parsing is needed. Only the files the pipeline uses were downloaded:

File	Rows x cols	Used for
Danmini_Doorbell/benign_traffic.csv	49,548 x 115	benign traffic, both scenarios
Danmini_Doorbell/mirai_attacks/scan.csv	107,685 x 115	attack traffic, Scenario 2
Danmini_Doorbell/gafgyt_attacks/combo.csv	59,718 x 115	sanity check (check_before_real_data.py)
Ennio_Doorbell/benign_traffic.csv	39,100 x 115	second device, Scenario 1

All files: 0 NaN, 0 Inf, identical column order. 20 columns (covariance and Pearson-correlation features) take negative values, and the dynamic range spans about 19 orders of magnitude (maximum 5.7e17), which is why evidence is computed in a calibrated space (section 6.2).

4.2 IoT-23 (second, independent dataset)

Stratosphere Laboratory, CTU Prague (Garcia et al., 2020). Only the labelled Zeek flow logs (conn.log.labeled) were downloaded, not the PCAPs. The parser handles the format quirks (space-separated last columns, 'det_label' vs 'detailed-label' headers, label case, '-' for missing values).

Capture	Flows	Role
CTU-IoT-Malware-Capture-34-1	23,145 (21,222 Mirai: DDoS + C&C;)	attack, plus device-A benign (1,923 flows)
CTU-Honeypot-Capture-5-1	1,374 benign (Amazon Echo)	device B, Scenario 1
CTU-Honeypot-Capture-4-1	452 benign (Philips HUE)	parsed, too small to use

21 features per flow: 8 numeric (duration, originator/responder bytes and packets, IP bytes, missed bytes), 3 protocol one-hots (tcp, udp, icmp) and 10 connection-state one-hots (S0, SF, REJ, ...).

4.3 Synthetic stream and other sources

- **Synthetic:** 20 Gaussian features (data/synth_stream.py), used as a fast regression check with known ground truth.
- **Fake N-BaIoT:** generated files with the real column names and dynamic range, only for tests on machines without the data.
- **CICIoT2023** was considered but not used: UNB requires a registration form.

4.4 The simulated reputation channel

meta['reputation'] is 1.0 for benign origin and 0.05 for attack origin. data/reputation.py degrades it in three independently controllable ways: Gaussian noise (sigma), a per-record flip rate (the feed reports the wrong verdict, as with a stale or poisoned blocklist) and lag (the verdict for record i arrives at record i + lag). Features, labels and phases are untouched, so every comparison against the oracle run stays controlled.

5. Data processing and features

5.1 Processing pipeline

- **Load.** N-BaIoT: CSV files, one row per packet window with 115 numeric statistics (no packet parsing needed). IoT-23: Zeek conn.log.labeled, parsed by data/load_iot23.py into 21 per-flow features (unset fields '-' become 0; the space-separated last columns and header variants are normalised).
- **Check.** Missing / infinite values, column order across files, sign of values (check_before_real_data.py, data_profile.py). No cleaning was needed: 0 NaN and 0 Inf in every file.
- **Label.** N-BaIoT labels come from the file (benign_traffic.csv = 0, attack files = 1). IoT-23 uses the capture's own label column (Malicious = 1).
- **Transform.** Signed log, sign(x) log(1 + |x|), then z-score. The z-score mean and standard deviation are fitted on training (calibration) data only and frozen; live data never moves them.
- **Split.** Benchmark: temporal per file and class, first 70% of rows in time order train, last 30% test, subsampled with an even stride. Scenarios: disjoint row ranges per phase, so no record is used twice.
- **Stream.** Scenario streams are 400 calibration records followed by phases of 20-record rounds (table below). The live experiment uses the real capture order instead.

5.2 Measured data quality

File	Rows	NaN	Inf	Negative columns	Max |value|
benign	49,548	0	0	20	5.67e+17
mirai_scan	107,685	0	0	0	5.60e+17
gafgyt_combo	59,718	0	0	20	5.67e+17
ennio_benign	39,100	0	0	20	5.88e+17

Capture	Flows	Malicious	Attack types
CTU-IoT-Malware-Capture-34-1	23,145	21,222	C&C; (6,706), PartOfAHorizontalPortScan (122), DDoS (14,394)
CTU-Honeypot-Capture-5-1	1,374	0	-
CTU-Honeypot-Capture-4-1	452	0	-

Table: raw file checks (N-BaIoT, top) and IoT-23 captures with attack types (bottom).

5.3 What each processing step does

Step	N-BaIoT std range	decades	IoT-23 std range	decades
raw values	0.006 - 2e+16	18.5	0.015 - 3.8e+06	8.4
signed log	0.005 - 14	3.5	0.011 - 2.6	2.4
signed log + z-score	1 - 1	0.0	1 - 1	0.0

Spread of per-feature standard deviations on the benchmark training split. Without the transform a single feature dominates every distance and the evidence signals collapse (section 6.2).

Scenario 2 stream layout (N-BaIoT; IoT-23 and synthetic use the same layout):

Phase	Records	Attack records	Purpose
calibration	400	120	labelled training set (30% attack)
pre_attack	400	0	clean benign steady state
slow_poison	600	600	real attack rows interpolated towards benign rows

Phase	Records	Attack records	Purpose
post_poison	400	400	attack fully mapped onto benign (undetectable from traffic)
recovery_check	200	200	raw attack again: was the model poisoned?

5.4 What the features are

N-BaIoT features were computed by the dataset authors with damped incremental statistics (the Kitsune extractor). A name such as HH_jit_L0.1_variance reads *aggregation_time window_statistic*. The five decay windows L5, L3, L1, L0.1 and L0.01 correspond to roughly 100 ms, 500 ms, 1.5 s, 10 s and 1 minute of history.

Aggregation	Grouped by	Statistics	Count
MI_dir	source MAC + IP (sending host)	weight, mean, variance	15
H	source IP (sending host)	weight, mean, variance	15
HH	source IP -> destination IP (channel)	weight, mean, std, magnitude, radius, covariance, pcc	35
HH_jit	channel inter-packet time (jitter)	weight, mean, variance	15
HpHp	source IP:port -> destination IP:port (socket)	weight, mean, std, magnitude, radius, covariance, pcc	35

Statistic	Meaning
weight	decayed packet count: how many packets recently
mean / variance / std	packet size (inter-packet time for HH_jit)
magnitude	sqrt(mean_out^2 + mean_in^2): traffic size in both directions
radius	sqrt(var_out^2 + var_in^2): spread in both directions
covariance, pcc	covariance and Pearson correlation of outbound vs inbound sizes

IoT-23 features, one row per Zeek connection (flow):

Feature	Meaning
duration	flow duration in seconds
orig_bytes	payload bytes sent by the originator
resp_bytes	payload bytes sent by the responder
missed_bytes	bytes Zeek missed (packet loss)
orig_pkts	packets sent by the originator
orig_ip_bytes	IP-level bytes sent by the originator
resp_pkts	packets sent by the responder
resp_ip_bytes	IP-level bytes sent by the responder
proto_tcp	one-hot: transport protocol TCP
proto_udp	one-hot: transport protocol UDP
proto_icmp	one-hot: transport protocol ICMP
state_S0	one-hot: SYN seen, no reply (scan / unanswered)
state_S1	one-hot: established, not terminated
state_SF	one-hot: normal establish and termination
state_REJ	one-hot: connection attempt rejected
state_RSTO	one-hot: established, originator aborted (RST)
state_RSTR	one-hot: responder sent RST
state_RSTOS0	one-hot: originator SYN then RST, no SYN-ACK

Feature	Meaning
state_SH	one-hot: originator SYN then FIN, no SYN-ACK
state_SHR	one-hot: responder SYN-ACK then FIN, no SYN
state_OTH	one-hot: no SYN seen, midstream traffic

5.5 Which features carry the signal

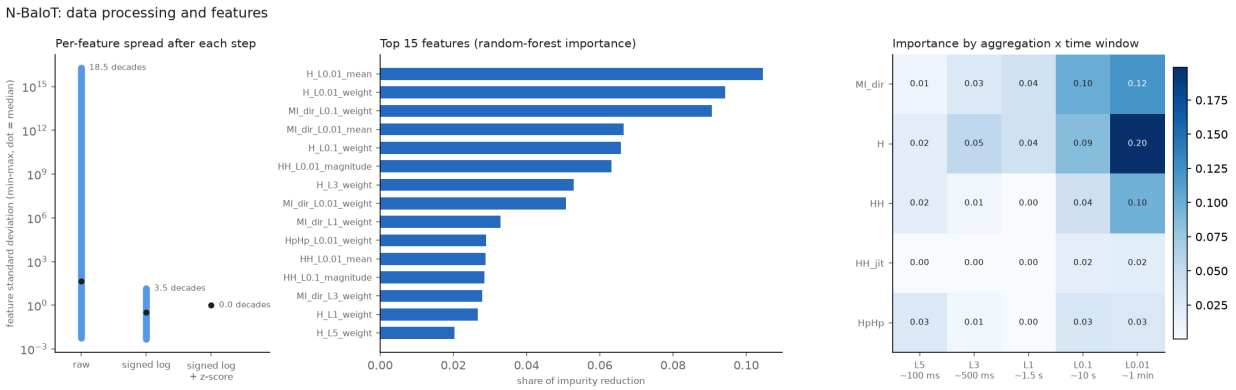


Figure: N-BaIoT processing and features. Left: per-feature spread after each step. Middle: top features by random-forest importance. Right: importance summed by aggregation and time window.

Rank	Feature	Meaning	Importance
1	H_L0.01_mean	mean packet size, source IP (sending host), window ~1 min	0.105
2	H_L0.01_weight	decayed packet count, source IP (sending host), window ~1 min	0.094
3	MI_dir_L0.1_weight	decayed packet count, source MAC+IP (sending host), window ~10 s	0.091
4	MI_dir_L0.01_mean	mean packet size, source MAC+IP (sending host), window ~1 min	0.066
5	H_L0.1_weight	decayed packet count, source IP (sending host), window ~10 s	0.066
6	HH_L0.01_magnitude	sqrt(mean_out^2 + mean_in^2), source->destination IP channel, window ~1 min	0.063
7	H_L3_weight	decayed packet count, source IP (sending host), window ~500 ms	0.053
8	MI_dir_L0.01_weight	decayed packet count, source MAC+IP (sending host), window ~1 min	0.051
9	MI_dir_L1_weight	decayed packet count, source MAC+IP (sending host), window ~1.5 s	0.033
10	HpHp_L0.01_weight	decayed packet count, source->destination socket (IP:port), window ~1 min	0.029

N-BaIoT: per-host (H, MI_dir) packet counts and sizes over the longest windows (~10 s, ~1 min) separate Mirai/Gafgyt from benign best: a bot sends many packets of uniform size. Jitter features contribute little.

Rank	Feature	Meaning	Importance
1	proto_tcp	one-hot: transport protocol TCP	0.266
2	proto_udp	one-hot: transport protocol UDP	0.145
3	orig_pkts	packets sent by the originator	0.103
4	orig_ip_bytes	IP-level bytes sent by the originator	0.097
5	state_SF	one-hot: normal establish and termination	0.090
6	orig_bytes	payload bytes sent by the originator	0.059
7	duration	flow duration in seconds	0.055

Rank	Feature	Meaning	Importance
8	resp_ip_bytes	IP-level bytes sent by the responder	0.043

IoT-23: transport protocol and packet/byte counts dominate (Mirai C&C; and DDoS flows are short, TCP, few packets). How the features are USED: every model receives the full transformed vector; the trust gate's evidence signals use the same frozen space (topology uses the first five features).

6. Algorithms

6.1 Online IDS and drift detection

The detector is River's pipeline StandardScaler | LogisticRegression, trained one record at a time with SGD. ADWIN (adaptive windowing, Bifet and Gavalda 2007) keeps a variable-length window of a 0/1 error stream and signals a change when two sub-windows have significantly different means. The shared monitor uses the frozen static arm, whose error rate is an unbiased measure of how far live traffic has moved from calibration. The optional second trigger runs ADWIN on each record's normalised distance from the calibration centre, ||z - c|| / sqrt(d), which catches benign-to-benign drift that never raises the error rate.

6.2 Calibrated feature space (trust boundary)

```
z = (slog(x) - mu_calib) / sd_calib        slog(x) = sign(x) * log(1 + |x|)
mu_calib, sd_calib: fit ONCE on calibration benign records, then frozen
```

The evidence signals are geometric, so features must be on comparable scales. On raw N-BaIoT values temporal stability evaluates to 1.7e-17 and five jitter variance columns carry 100% of every cosine, so nothing is ever authorized. The signed log keeps the sign of correlation features. Fitting on calibration data only means the attacker can never move the reference frame.

6.3 Traffic evidence

```
shift        = mean(window_z) - trusted_baseline
attribution  = (cos(shift, reference_direction) + 1) / 2      (0.5 if none yet)
reference_direction <- 0.7 * reference_direction + 0.3 * shift  (after authorization)
topology     = H(histogram of 5 features, 10 bins) / log(9)     (normalised entropy)
temporal     = 1 / (1 + ||mean(window_z) - mean(previous_window_z)||)
trust score  T = 0.4 * attribution + 0.3 * topology + 0.3 * temporal
```

Attribution asks whether this drift continues a direction that was already authorized; topology rewards varied, device-like behaviour over a narrow repeated channel; temporal stability rewards gradual change, which is exactly what a slow attacker maximises, so it is not sufficient on its own. Weights are normalised to sum to 1 (negative or all-zero weights are rejected).

6.4 One-shot trust gate (Review 2)

```
r = mean reputation of the 20-record window
if r < 0.3:            REFUSE  (out-of-band veto)
elif T >= 0.55:        AUTHORIZE
else:                  REFUSE  (weak traffic evidence)
```

6.5 Elastic Weight Consolidation baseline

After calibration the logistic-regression weights w* are frozen and the diagonal Fisher information F is estimated on calibration inputs (p(1-p) z^2, normalised to a maximum of 1). After each SGD step every weight is pulled back: w <- w - min(1, eta * lambda * F) (w - w*). This penalises moving important weights, the standard defence against catastrophic forgetting. Only the LR weights are anchored; the scaler keeps adapting.

6.6 Quarantine extension (new)

Sequential reputation test. Each record's reputation is a vote (benign if > 0.5). The feed is assumed wrong on at most e = 30% of records, so under 'benign episode' a benign vote has probability 1 - e and under 'attack episode' probability e. Wald's sequential probability ratio test accumulates the log-likelihood ratio over the whole drift episode:

```
step  = ln((1 - e) / e) = 0.847                per vote: LLR += step (benign) or -= step
A     = ln((1 - beta) / alpha) = +4.60         upper bound; LLR is clamped at A
B     = ln(beta / (1 - alpha)) = -4.60         lower bound; reaching B condemns the episode
alpha = beta = 0.01                            target false-condemn / miss rates
```

Clamping at A stops a long benign prefix (a lagged feed) from banking credit an attack would first have to burn off. Condemnation is sticky until a new episode starts.

Provisional updates, hold and rollback.

```
for each round:
    commit updates older than the hold (20 rounds)
    feed this window's votes to the SPRT (if drift-active or updates are held)
    if episode condemned and updates are held:
        restore the checkpoint taken before the episode's earliest held update
        replay held updates of other episodes on top (fresh checkpoints)
    if not drift-active:       nothing is proposed
    elif condemned:            refuse
    elif LLR <= 0:             hold (evidence not yet favourable)
    elif T >= 0.45:            save checkpoint, apply update PROVISIONALLY
    else:                      refuse
a new drift run starts a new episode only when nothing is held
```

Because a wrong acceptance is now reversible, the provisional threshold is 0.45, 0.1 below the one-shot gate's 0.55 (set before seeing results). The hold was first 5 rounds; the 600-record-lag sweep showed that a hold shorter than the feed's lag lets poison commit before the evidence arrives, so it was raised to 20 rounds (400 records). That is the only setting changed after seeing results, and the 5-round hold is kept as an ablation.

6.7 How the models are trained

Scenario pipeline (every arm starts from the same trained model):

Component	Setting	Value
Detector	model	River StandardScaler | LogisticRegression
	optimizer / learning rate	SGD, constant 0.01 (intercept 0.01)
	loss / regularisation	log loss, no L1/L2
	initial training	400 labelled calibration records (30% attack), one pass, shuffled (seed 42)
	later training	one learn_one per record of an AUTHORIZED 20-record update, label 'benign'
Drift trigger	ADWIN	delta 0.002, clock 32 (River defaults)
	drift-active window	15 rounds after each change point
	feature-drift trigger	ADWIN on ||z - c|| / sqrt(d) per record (optional)
Gates	weights / thresholds	0.4 / 0.3 / 0.3; trust 0.55; veto 0.3
Quarantine	SPRT / hold	feed error 0.3, alpha = beta = 0.01; provisional 0.45; hold 20 rounds
EWC	lambda / Fisher	50; diagonal Fisher on calibration inputs, max 1
Evaluation	protocol	prequential: predict, score, then (maybe) learn

Benchmark (section 9): every model gets the same preprocessed training split (signed log + z-score fitted on it), is trained once, and is scored on the later test split; 3 repeats with different seeds.

Benchmark model	Library	Hyperparameters
Logistic regression	scikit-learn	lbfgs, max_iter 2000
Linear SVM	scikit-learn	LinearSVC, max_iter 5000
Decision tree	scikit-learn	min_samples_leaf 5

Benchmark model	Library	Hyperparameters
Random forest	scikit-learn	100 trees, min_samples_leaf 2
Gradient boosting	scikit-learn	HistGradientBoosting, defaults
k-NN	scikit-learn	k = 5
Gaussian naive Bayes	scikit-learn	defaults
MLP	scikit-learn	hidden (64, 32), early stopping, max_iter 300
Isolation forest	scikit-learn	200 trees, fit on benign only
Autoencoder	scikit-learn	MLPRegressor (64, 16, 64) on benign; alarm above the 99th percentile of benign training error (Meidan et al.)
Online LR / Hoeffding tree / HAT / ARF (5 trees) / Gaussian NB	River	defaults; one pass over 12,000 shuffled training rows

7. Experimental setup

Every stream begins with 400 labelled calibration records (30% attack) on which all arms are trained identically. The rest is processed in 20-record rounds.

Scenario	Phases (records)	What a good system does
2 - Slow poisoning	pre-attack benign (400); slow poison: real attack rows interpolated towards real benign rows in signed-log space (600); post-poison: attack held in the benign region (400); recovery check: raw attack again (200)	Refuse the poisoning updates, so the obvious repeat attack is still detected (recovery-check accuracy 1.0).
1 - Genuine drift	device A benign (400); gradual blend A to B (600); device B benign (400)	Accept the benign drift, so accuracy after the shift stays high.

Metrics. Prequential accuracy and F1 (attack = positive), and the windowed accuracy in each scenario's decisive phase: *recovery_check* for Scenario 2 and *post_drift* for Scenario 1. The sweeps also count *poisoned updates kept*: authorized updates whose window contained attack records and that were never rolled back. F1 is 0 in Scenario 1 because that scenario has no attack records after calibration; accuracy is the metric there.

Seeds. Seeds reshuffle calibration rows (N-BaIoT, IoT-23) or regenerate the world (synthetic). The default run uses the loaders' fixed seed 42.

8. Results

8.1 Slow poisoning: accuracy on the obvious repeat attack

Arm	N-BaIoT	IoT-23	Synthetic
Static (never updates)	1.000	1.000	1.000
Ungated (learns everything)	0.200	0.000	0.200
Traffic-only gate	0.200	0.300	0.200
Full gate (one-shot veto)	1.000	1.000	1.000
EWC (lambda = 50)	1.000	1.000	1.000
Quarantine gate (new)	1.000	1.000	1.000
Drift-active rounds	24	32	37

Table 1. Recovery-check accuracy, oracle reputation feed, seed 42.

Learning from every drift (ungated) or trusting traffic evidence alone (traffic-only) lets the attacker teach the model that the attack is normal: on real N-BaIoT both detect only 20% of the repeat attack, on IoT-23 0% and 30%. Both reputation-aware gates refuse every poisoning update and keep 1.000. EWC also survives here, but at a cost shown in 8.2.

Arm	N-BaIoT	IoT-23	Synthetic
Static (never updates)	1.000 +/- 0.000	1.000 +/- 0.000	1.000 +/- 0.000
Ungated (learns everything)	0.200 +/- 0.000	0.000 +/- 0.000	0.170 +/- 0.035
Traffic-only gate	0.200 +/- 0.000	0.265 +/- 0.032	0.171 +/- 0.035
Full gate (one-shot veto)	1.000 +/- 0.000	1.000 +/- 0.000	1.000 +/- 0.000
EWC (lambda = 50)	0.897 +/- 0.155	1.000 +/- 0.000	1.000 +/- 0.000
Quarantine gate (new)	1.000 +/- 0.000	1.000 +/- 0.000	1.000 +/- 0.000

Table 2. Recovery-check accuracy over 10 seeds, mean +/- 95% CI.

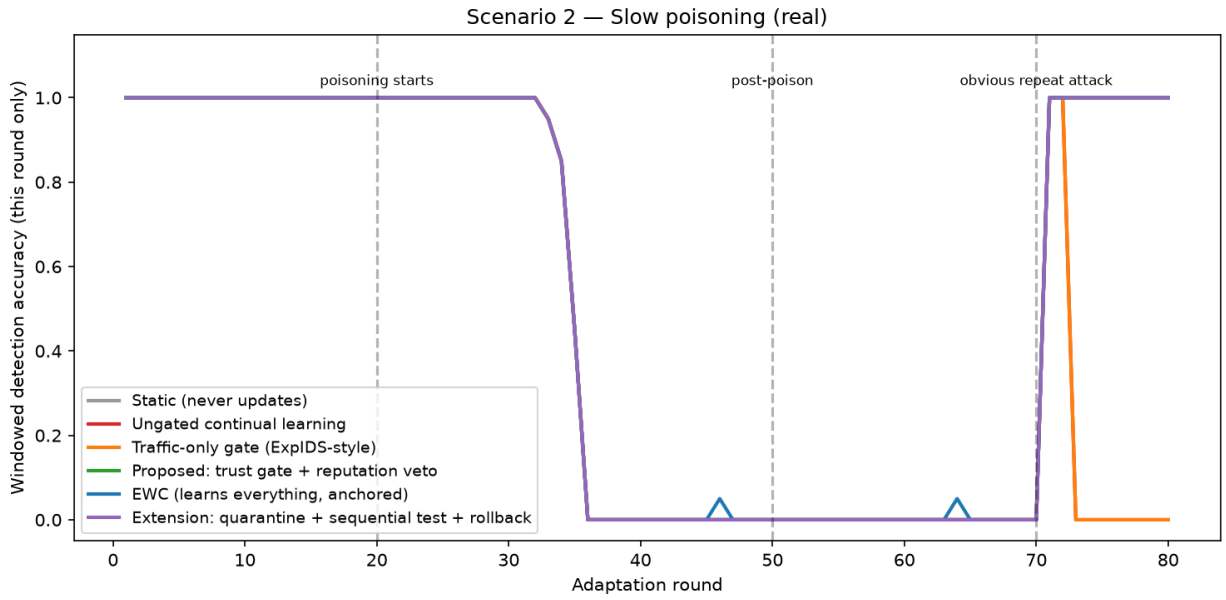


Figure 2. Scenario 2 on real N-BaIoT: windowed accuracy per round. Only the final 'obvious repeat attack' phase separates the arms.

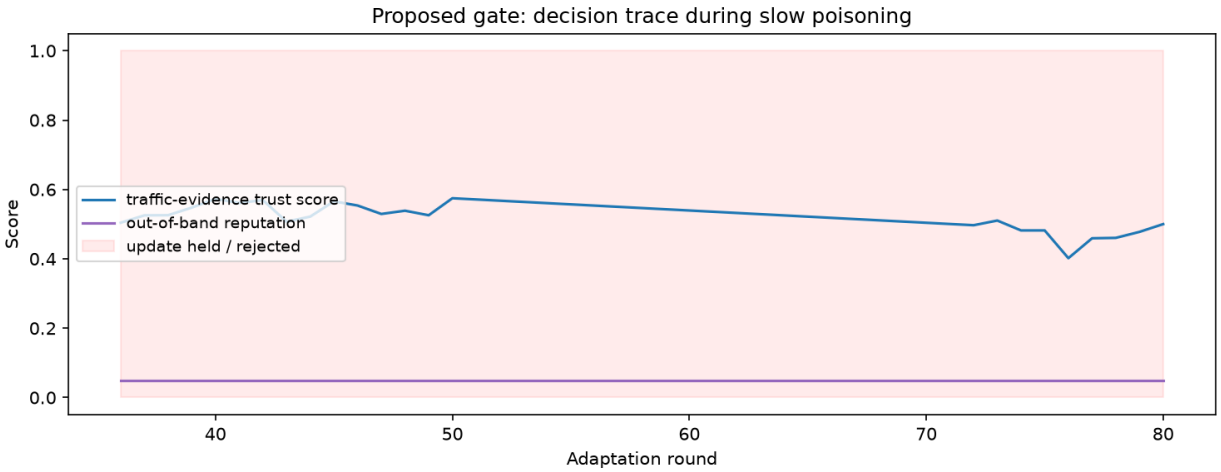


Figure 3. One-shot gate decision trace (drift-active rounds): the traffic trust score (blue) looks acceptable while reputation (purple) stays low, so the veto refuses (red shading).

8.2 Genuine drift: accuracy after a benign device shift

Arm	N-BaIoT + FD	IoT-23	IoT-23 + FD	Synthetic
Static (never updates)	0.993	0.860	0.860	0.000
Ungated (learns everything)	0.995	0.978	0.988	1.000
Traffic-only gate	0.993	0.978	0.905	1.000
Full gate (one-shot veto)	0.993	0.978	0.905	1.000
EWC (lambda = 50)	0.915	0.760	0.760	0.003
Quarantine gate (new)	0.995	0.975	0.975	1.000
Drift-active rounds	15	15	34	30

Table 3. Post-drift accuracy (FD = feature-drift trigger on). Without FD, real N-BaIoT detects no drift at all: a null result, all arms 0.993.

On IoT-23 the frozen static model falls to 0.860 after the device shift, while the gated learners recover to about 0.98: adaptation is worth having. EWC pays for its poisoning resistance by not adapting (0.760). With the feature-drift trigger, the one-shot gate refuses most real benign drift (IoT-23 0.905; on N-BaIoT its trust score peaks at 0.543, just under 0.55), while the quarantine gate's lower provisional threshold accepts it (0.975 on IoT-23, 0.995 on N-BaIoT).

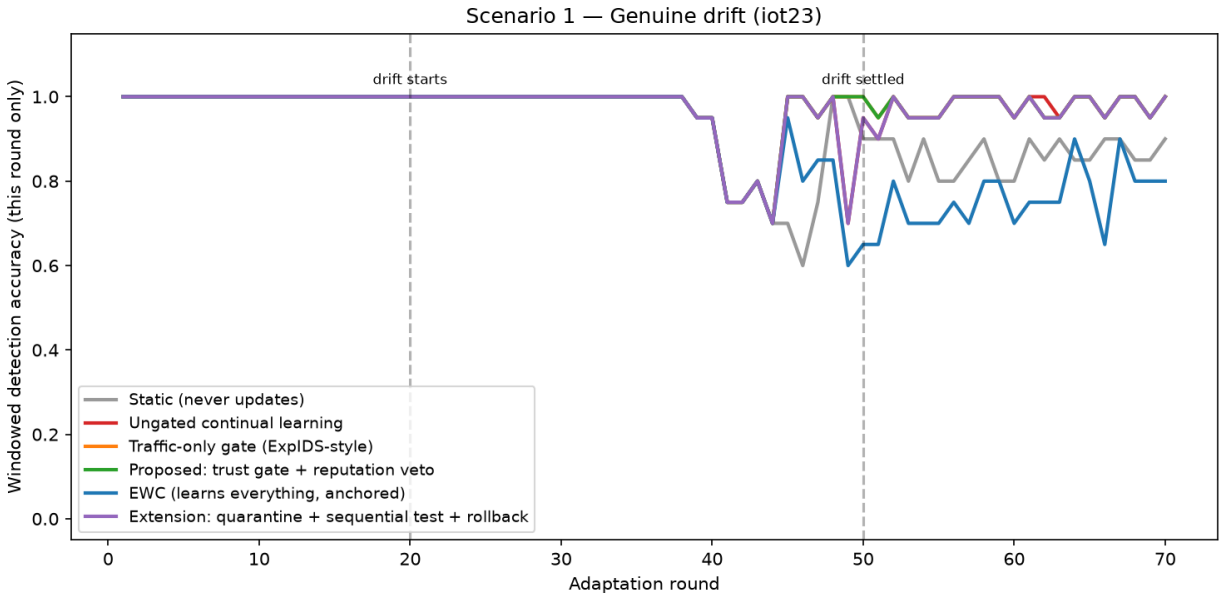


Figure 4. Scenario 1 on IoT-23: static drops after the shift; learners that adapt keep high accuracy, EWC does not.

8.3 Why the overall accuracy looks low (about 0.55)

The headline 'overall accuracy' is a running average over the whole Scenario-2 stream, and that stream is *designed* to be undetectable for 1,000 of its 1,600 evaluated records:

Arm	normal traffic	attack morphing to benign	attack disguised as benign	raw repeat attack	overall
Static (never updates)	1.000	0.475	0.000	1.000	0.553
Ungated (learns everything)	1.000	0.475	0.000	0.200	0.453
Traffic-only gate	1.000	0.475	0.000	0.200	0.453
Full gate (one-shot veto)	1.000	0.475	0.000	1.000	0.553
Quarantine gate (new)	1.000	0.475	0.000	1.000	0.553

Accuracy per phase, real N-BaIoT. In the two middle phases the attack has been moved onto the benign feature manifold on purpose; no traffic classifier can detect it there (every arm scores ~0), and that is the point of the scenario: the question is whether the model gets *poisoned* while it cannot see the attack. The last column is what the old report showed.

Measured the standard way, accuracy is high: on clean, later held-out traffic our detector scores 0.995 (F1 0.995) on N-BaIoT; on normal traffic and on the raw repeat attack the protected arms score 1.000; and on 21,000 live flows of a real capture the gated systems score 0.997. Reporting the per-phase table rather than one average is the honest fix.

9. Comparison with other models

9.1 Detector benchmark on N-BaIoT

Temporal split; train 30,000 rows (15,000 attack), test 12,000 later rows (6,000 attack), 115 features, mean of 3 repeats. us/rec = latency to classify one record as it arrives.

Model	Family	Acc.	F1	AUC	FPR	us/rec	Train s	KB
Logistic regression	batch	1.0000	1.0000	1.0000	0.0000	230	0.06	2
Random forest	batch	1.0000	1.0000	1.0000	0.0000	10678	2.02	110
Gradient boosting (HistGB)	batch	0.9999	0.9999	1.0000	0.0001	2038	0.70	431
Hoeffding tree (River)	online	0.9999	0.9999	0.9999	0.0001	28	2.29	878
Hoeffding adaptive tree (River)	online	0.9999	0.9999	0.9999	0.0001	38	3.60	887
Linear SVM	batch	0.9999	0.9999	1.0000	0.0002	206	0.29	1
MLP (64-32)	batch	0.9997	0.9997	0.9998	0.0000	249	1.69	231
k-NN (k=5)	batch	0.9997	0.9997	0.9998	0.0000	4193	0.00	27188
Adaptive random forest (River)	online	0.9995	0.9995	1.0000	0.0009	32	4.77	462
Online logistic regression (River)	online	0.9994	0.9994	0.9993	0.0003	24	1.14	7
Decision tree	batch	0.9987	0.9987	1.0000	0.0027	184	0.31	2
Gaussian naive Bayes	batch	0.9978	0.9978	0.9976	0.0037	308	0.04	4
Gaussian naive Bayes (River)	online	0.9971	0.9971	0.9976	0.0058	277	0.54	865
Online LR, 400 calibration rows (OURS)	online	0.9954	0.9954	0.9944	0.0072	24	0.04	7
Autoencoder (benign only, N-BaIoT paper)	unsup.	0.9931	0.9931	0.9976	0.0139	241	5.97	408
Isolation forest (benign only)	unsup.	0.9584	0.9601	0.9928	0.0832	23501	0.26	1773

Detector benchmark on N-BaIoT: temporal train/test split

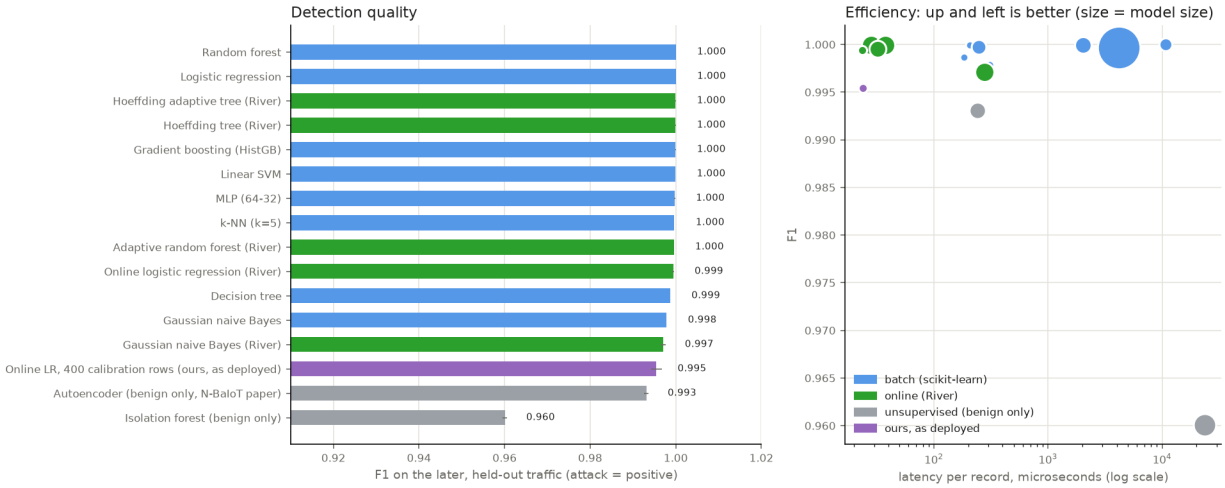


Figure: detector benchmark on N-BaIoT. Left: F1. Right: F1 against per-record latency (log), bubble size = model size.

Unseen device. False alarms on Ennio_Doorbell benign traffic, a device no model trained on: best k-NN (k=5) 0.000; ours 0.358. Trained on only 400 rows, our detector generalises worst to a new device. This is exactly the situation the continual-learning loop exists for (Scenario 1), and why it must adapt safely.

9.2 Detector benchmark on IoT-23

Temporal split; train 8,623 rows (6,000 attack), test 4,126 later rows (3,000 attack), 21 features, mean of 3 repeats. us/rec = latency to classify one record as it arrives.

Model	Family	Acc.	F1	AUC	FPR	us/rec	Train s	KB
Gradient boosting (HistGB)	batch	0.9891	0.9926	0.9864	0.0400	2200	0.22	355
Logistic regression	batch	0.9886	0.9922	0.9874	0.0409	231	0.03	1
MLP (64-32)	batch	0.9886	0.9922	0.9897	0.0409	244	0.52	90
Random forest	batch	0.9886	0.9922	0.9916	0.0400	10624	0.21	452
Adaptive random forest (River)	online	0.9885	0.9922	0.9965	0.0411	54	2.46	1360
Linear SVM	batch	0.9884	0.9921	0.9863	0.0417	204	0.40	1
k-NN (k=5)	batch	0.9884	0.9921	0.9798	0.0409	800	0.00	1483
Decision tree	batch	0.9879	0.9917	0.9774	0.0400	181	0.01	6
Gaussian naive Bayes	batch	0.9869	0.9911	0.9781	0.0471	345	0.00	1
Online logistic regression (River)	online	0.9864	0.9908	0.9894	0.0488	6	0.18	2
Hoeffding adaptive tree (River)	online	0.9860	0.9905	0.9789	0.0512	14	0.76	576
Hoeffding tree (River)	online	0.9855	0.9902	0.9720	0.0530	6	0.42	427
Online LR, 400 calibration rows (OURS)	online	0.9850	0.9898	0.9870	0.0542	5	0.01	2
Isolation forest (benign only)	unsup.	0.9230	0.9497	0.9270	0.2821	23305	0.20	1867
Autoencoder (benign only, N-BaIoT paper)	unsup.	0.5755	0.5150	0.6518	0.0207	232	0.72	125
Gaussian naive Bayes (River)	online	0.2729	0.0000	0.9717	0.0000	124	0.08	158

Detector benchmark on IoT-23: temporal train/test split

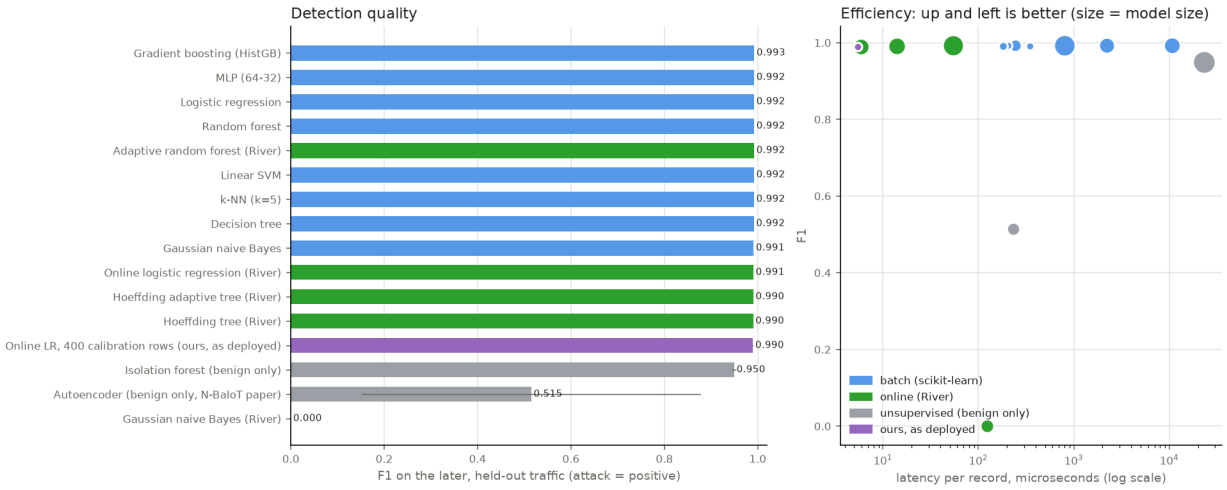


Figure: detector benchmark on IoT-23. Left: F1. Right: F1 against per-record latency (log), bubble size = model size.

Reading the benchmark. On clean data the detection problem is easy for supervised models (F1 0.99-1.00). Our online LR is within 0.5 points of the best while being 10-1000x faster per record than the batch ensembles and tiny (7 KB). Unsupervised models trained on benign traffic only (isolation forest, the N-BaIoT paper's autoencoder) are clearly weaker. River's naive Bayes collapses on IoT-23 (it predicts everything benign at the 0.5 threshold, AUC 0.97).

9.3 Every defence on every base learner

Repeat attack after poisoning (each cell: N-BaIoT / IoT-23)

Base learner	static	ungated	traffic-only	full gate	quarantine
Logistic regression (ours)	1.00 / 1.00	0.20 / 0.00	0.20 / 0.30	1.00 / 1.00	1.00 / 1.00
Hoeffding tree	1.00 / 0.00	0.70 / 0.00	1.00 / 0.00	1.00 / 0.00	1.00 / 0.00
Hoeffding adaptive tree	1.00 / 0.00	0.70 / 0.00	1.00 / 0.00	1.00 / 0.00	1.00 / 0.00
Adaptive random forest	1.00 / 1.00	0.30 / 0.00	0.20 / 0.12	1.00 / 1.00	1.00 / 1.00
Gaussian naive Bayes	1.00 / 1.00	1.00 / 0.01	1.00 / 0.80	1.00 / 1.00	1.00 / 1.00

Benign drift (feature-drift trigger), post-drift (each cell: N-BaIoT / IoT-23)

Base learner	static	ungated	traffic-only	full gate	quarantine
Logistic regression (ours)	0.99 / 0.86	0.99 / 0.99	0.99 / 0.90	0.99 / 0.90	0.99 / 0.97
Hoeffding tree	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00
Hoeffding adaptive tree	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00
Adaptive random forest	1.00 / 0.79	1.00 / 1.00	1.00 / 0.98	1.00 / 0.98	1.00 / 1.00
Gaussian naive Bayes	1.00 / 0.96	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00

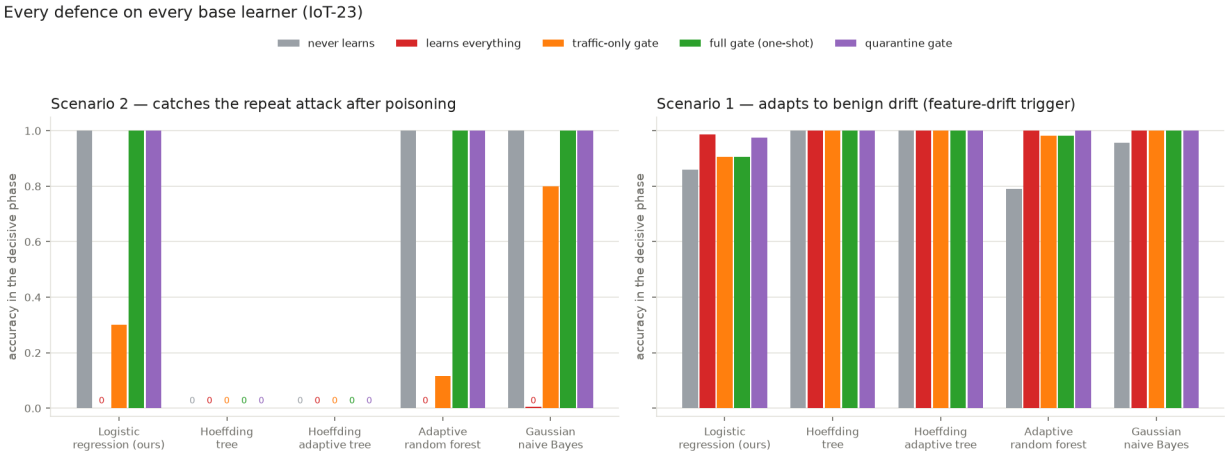


Figure: every defence on every base learner (IoT-23). Zero-height bars are marked '0'.

Result. The gate's protection does not depend on the learner. Wherever the detector itself works (static = 1.00), both reputation-aware gates keep the repeat attack at 1.00, while learning everything is poisoned (0.00-0.70) in 7 of those 8 learner/dataset pairs; the traffic-only gate fails in 5 of them. Two honest exceptions: Hoeffding trees on IoT-23 never split on 400 calibration flows, so they detect nothing even before poisoning (no gate can fix a detector that never learned); and on N-BaIoT naive Bayes moves so little that even learning everything leaves it at 1.00 (on IoT-23 it is poisoned to 0.01 like the others). On benign drift the quarantine gate matches or beats the one-shot gate on every learner (IoT-23 logistic regression 0.97 vs 0.90).

10. Live experiment

Setup. The IoT-23 capture CTU-IoT-Malware-Capture-34-1 (23,145 real flows recorded over 24 hours; Mirai C&C; and DDoS mixed with benign traffic, including a DDoS burst of about 13,000 flows at hour 17.5) is replayed in its original time order and timing, compressed 600x, as a network tap would deliver it. Nothing is constructed: no phases, no interpolation. The first 2,000 flows in time train the detector; the remaining flows are unseen and arrive on the wall clock. All six systems process every 20-flow round as it arrives, with the feature-drift trigger on. Labels are used only to score predictions afterwards.

System	Accuracy	Precision	Recall	F1	FPR	Updates applied
Static (never updates)	0.9971	0.9981	0.9987	0.9984	0.0238	0
Ungated (learns everything)	0.1343	1.0000	0.0658	0.1234	0.0000	139
Traffic-only gate	0.2982	0.9923	0.2445	0.3923	0.0238	43
Full gate (one-shot veto)	0.9971	0.9981	0.9987	0.9984	0.0238	8
EWC (lambda = 50)	0.3233	0.9920	0.2719	0.4268	0.0277	139
Quarantine gate (new)	0.9971	0.9981	0.9987	0.9984	0.0238	6

21,140 live flows (19,588 malicious), 139 drift-active rounds.

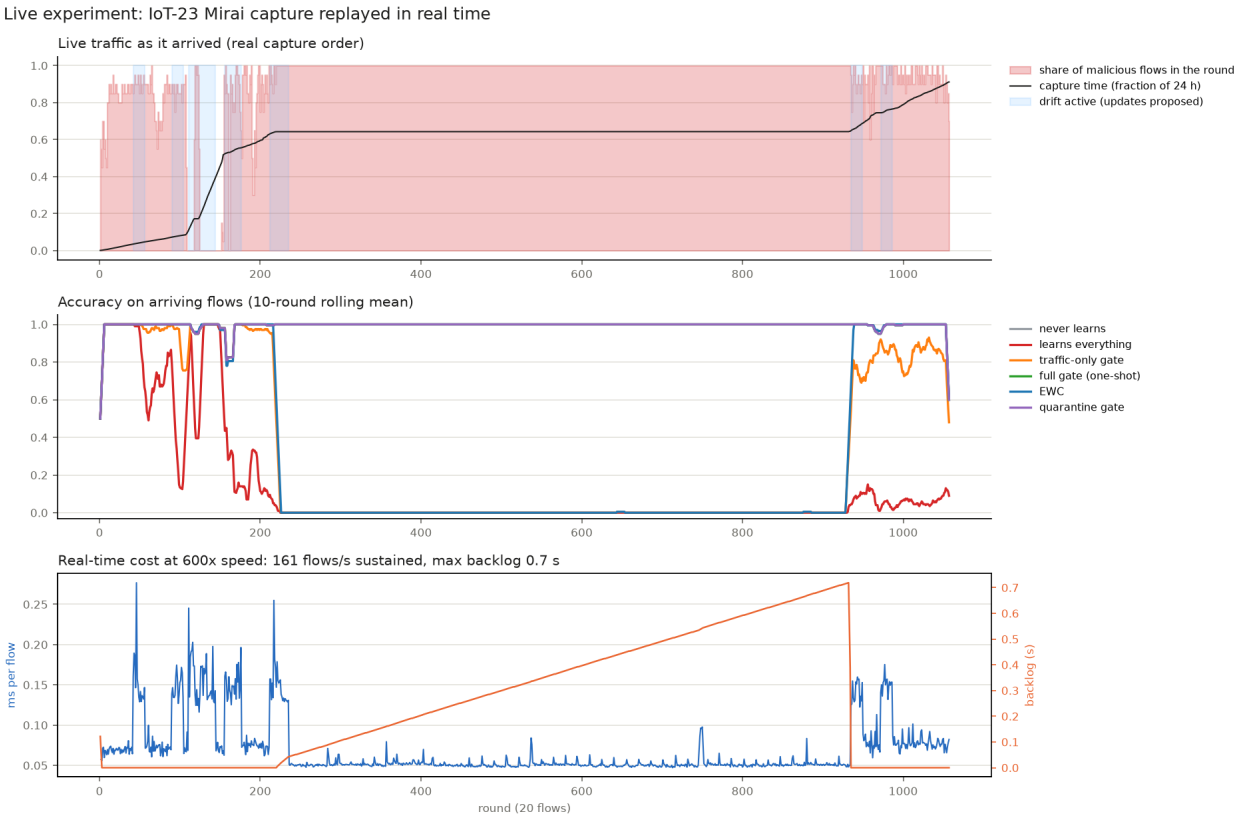


Figure: live run. Top: traffic as it arrived and when drift was active. Middle: rolling accuracy per system. Bottom: processing time per flow and backlog behind real time.

Result. On real traffic in real order, naive self-learning destroys the detector: learning every drift-triggered update drops accuracy to 0.134 (recall 0.066), the traffic-only gate to 0.298, EWC to 0.323. Both reputation-aware gates keep 0.997 while still applying 8 (one-shot) and 6 (quarantine) benign updates.

Real-time cost. At 600x the pipeline kept up with arrival (161 flows/s needed); the worst backlog, during the DDoS burst, was 0.72 s. Processing all six systems takes a median 0.052 ms per flow (95th percentile 0.153 ms). Without pacing the same run processes 13,766 flows/s (21,140 flows in 1.5 s).

Live sensor path. POST /api/live/ingest accepts raw Zeek conn.log lines (the #fields header plus rows) and scores them with a detector trained on the capture's first 2,000 flows; experiments/live_feed.py tails a growing conn.log (--follow), as written by a running Zeek sensor. Measured by feeding real logs: 10-12 us per flow; running accuracy 0.945 on the first 3,000 flows of the Mirai capture and 0.832 on 1,000 flows of the Amazon Echo honeypot, a device the detector never saw (17% false alarms; consistent with the unseen-device result in section 9.1). The ingest path is detection-only: the trust gate needs a reputation feed, which a raw sensor does not provide.

Limits. This is a real-time replay of a real capture, not packets captured on this machine; reputation is still simulated from the labels.

11. Robustness and the quarantine extension

11.1 One-shot veto under an imperfect feed

Full gate under an imperfect reputation channel

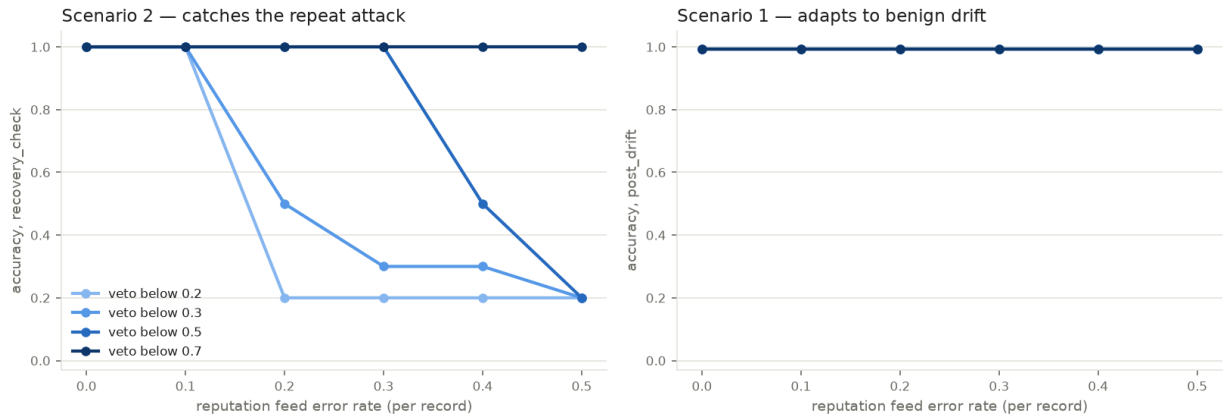


Figure 5. One-shot full gate vs feed error rate, one line per veto threshold (real N-BaIoT).

The default veto (0.3) holds only up to 10% feed error: at 20% the attack window's mean reputation crosses 0.3 often enough for poison to get in (0.50 on N-BaIoT, 0.60 on IoT-23). A 0.5 veto holds to 30-40%; 0.7 holds longer but starts blocking benign drift. A fixed per-window threshold is fragile.

11.2 EWC strength

EWC trades adaptation for stability; the gate needs neither trade

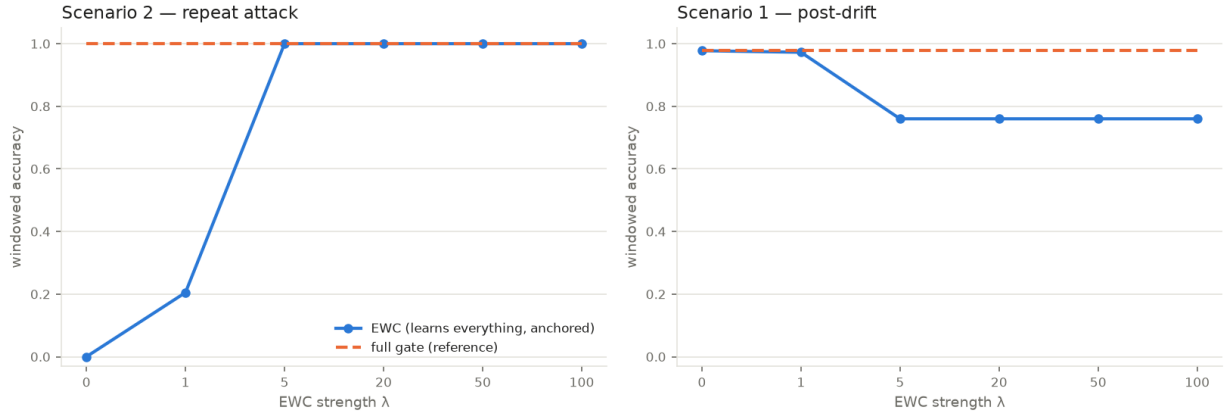


Figure 6. EWC lambda sweep on IoT-23: survives poisoning from lambda = 5 but post-drift accuracy falls to 0.76, vs 0.98 for the gate.

11.3 Quarantine vs one-shot gate

Feed	Metric	veto<0.3	veto<0.5	quarantine	no rollback	5-round hold
20% wrong	repeat-attack acc.	0.50 / 0.60 / 0.50	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00
	poison kept	2 / 6 / 3	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0
30% wrong	repeat-attack acc.	0.30 / 0.50 / 0.30	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00
	poison kept	3 / 5 / 7	1 / 1 / 1	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0
40% wrong	repeat-attack acc.	0.30 / 0.50 / 0.30	0.50 / 1.00 / 0.50	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00
	poison kept	3 / 6 / 9	2 / 2 / 2	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0

Feed	Metric	veto<0.3	veto<0.5	quarantine	no rollback	5-round hold
300 late	poison kept	0 / 1 / 0	0 / 1 / 0	0 / 0 / 0	0 / 4 / 0	0 / 0 / 0
600 late	poison kept	1 / 1 / 2	1 / 1 / 2	0 / 0 / 0	1 / 14 / 6	1 / 14 / 5

Table 4. Seed 42; each cell is N-BaIoT / IoT-23 / Synthetic. 'Poison kept' = authorized updates on attack windows never rolled back. Read from results/sweep_quarantine_*.json.

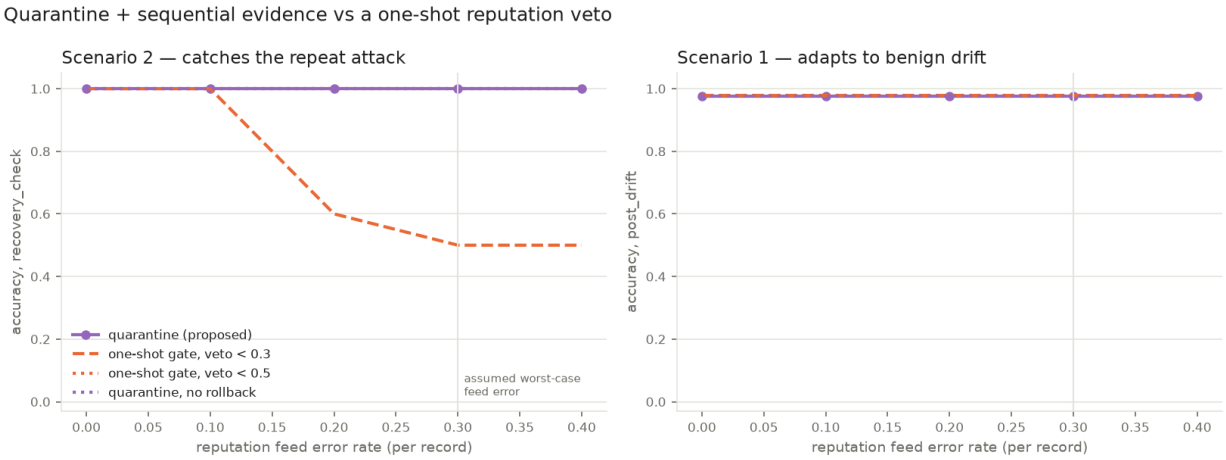


Figure 7. Quarantine sweep on IoT-23 (single seed).

Over 5 seeds (measured separately, benign drift with the feature-drift trigger on the real datasets):

Feed error	Repeat attack: quarantine vs one-shot (N-BaIoT / IoT-23 / synth)	Benign drift: quarantine vs one-shot
20%	1.000 / 1.000 / 1.000 vs 0.42 / 0.62 / 0.40	0.995 / 0.976 / 1.000 vs 0.997 / 0.960 / 1.000
30%	1.000 / 1.000 / 1.000 vs 0.22 / 0.32 / 0.20	0.995 / 0.976 / 1.000 vs 0.997 / 0.960 / 1.000
40%	0.88 / 1.000 / 1.000 vs 0.22 / 0.32 / 0.20	0.996 / 0.899 / 0.398 vs 0.997 / 0.960 / 1.000

Table 5. 5-seed means, hand-entered from a separate run.

Reading the ablations. The sequential test is what makes the gate robust to a *wrong* feed (zero poison at 20-40% error, where the one-shot veto at 0.3 keeps 2-9 updates and even 0.5 keeps 1-2). Rollback with a hold longer than the feed lag is what handles a *late* feed (without rollback 4-14 poisoned updates stay on IoT-23). The lower provisional threshold is what admits real benign drift. **Limit:** above the assumed 30% feed error the SPRT's guarantee is gone; it starts condemning benign drift (synthetic 0.40) and on N-BaIoT one seed in five lets the attack through.

12. Related work and novelty

A targeted (not systematic) literature search found every building block already published:

Idea	Prior work
Slow ('boiling frog') poisoning of self-updating detectors	Rubinstein et al., ANTIDOTE (IMC 2009); 'Can't Boil This Frog' (2020)
Poisoning that fakes or hides concept drift	Korycki and Krawczyk, adversarial concept drift detection (2020)
Drift detection and adaptation for security classifiers	Transcend (USENIX Sec 2017), CADE (USENIX Sec 2021), OWAD (NDSS 2023), INSOMNIA (2021), ExpIDS (2025)
Reject updates a trusted reference contradicts	RONI (Nelson et al. 2008); FLTrust trusted root data (NDSS 2021)
Trust-weighted updates and rollback after poisoning	Trust-aware federated IDS (AGAT-FL, 2025); Crab (2024), FedRecover (2023)
Sequential tests over IDS feedback	Zhu et al., collaborative IDS with sequential hypothesis testing (2010)

What was not found: a single, non-federated, self-labelling online IoT IDS that (a) gates its own drift-triggered updates on an out-of-band channel independent of the traffic features, and (b) treats that channel as sequential, fallible evidence about a whole drift episode, with explicit assumption-stated error rates, applying updates provisionally and rolling them back when the evidence turns. Federated work weighs *clients*; RONI and FLTrust need trusted *labelled* data; drift-adaptation work (OWAD, CADE, Transcend) does not model an adversary who manufactures the drift.

The defensible novelty claim is therefore the **combination** and the **episode-level sequential treatment of the independent channel**, not any single component.

13. Software, packages and backend

13.1 Packages (requirements.txt, pinned)

Package	Version	Used for
Python	3.12	runtime (virtual environment .venv)
river	0.26.1	online logistic regression, StandardScaler, ADWIN, metrics
numpy	2.5.4	vectors, evidence geometry
pandas	3.0.6	CSV / Zeek log loading, tables in the app
scipy	1.18.1	t-distribution confidence intervals in sweeps
scikit-learn	1.9.1	benchmark models, random-forest feature importance
matplotlib	3.11.2	static figures in results/
streamlit	1.66.0	interactive demo app
altair	6.3.0	interactive charts in the app
fastapi	0.144.0	backend REST API and SSE streams
uvicorn[standard]	0.54.0	ASGI server for the backend
httpx	0.28.1	app-to-backend client, API tests
pytest	9.1.1	test suite
reportlab	4.2.5	building this PDF (docs/build_report.py)

13.2 Backend

backend/server.py is a FastAPI application. Runs execute in worker threads; every round is logged to the console, logs/backend.log and an in-memory ring buffer that /api/logs/stream tails. Finished runs are saved to results/runs/ and reloaded on restart. Requests are validated with Pydantic (weights not all zero, feed error in (0, 0.5), ...). TRUST_IDS_TOKEN enables API-key auth (X-API-Key header or ?token=); TRUST_IDS_MAX_RUNS (default 2) caps concurrent runs and returns HTTP 429 above it. Port 8765 is used because 8000 is taken on the development machine.

Endpoint	Purpose
GET /	live log console
GET /api/health	status, dataset availability, auth, run limits
GET /api/results/{scenario}/{tag}	precomputed log (real, iot23, synth)
POST /api/runs	start a run: scenario, source, gate, reputation_noise, quarantine, seed, feature_drift, EWC, pace
GET /api/runs, /api/runs/{id}	list runs / run status and events
GET /api/runs/{id}/events	Server-Sent Events, one per round
GET /api/logs/stream	Server-Sent Events of all server logs

14. Visualizations

Colours are consistent everywhere: grey static, red ungated, orange traffic-only, green full gate, blue EWC, purple quarantine. PROJECT_GUIDE.txt explains each in more detail.

Visualization	What it shows / how to read it
scenario2_slow_poisoning_<tag>.png	Windowed accuracy per round in Scenario 2. Judge the last phase (obvious repeat attack): poisoned systems score low, protected ones 1.0.
scenario1_genuine_drift_<tag>.png	Same for the benign device shift; adapting systems stay high, static may drop.

Visualization	What it shows / how to read it
gate_decision_trace_<tag>.png	Trust score (blue) and reputation (purple) on drift-active rounds; red shading = update held or refused.
sweep_reputation_<tag>.png	One-shot gate vs feed error, one line per veto threshold (light to dark = low to high). Left: attack resistance, right: benign adaptation.
sweep_ewc_<tag>.png	EWC lambda vs both scenarios, gate as dashed reference: EWC trades adaptation for stability.
sweep_quarantine_<tag>.png	Quarantine (purple) vs one-shot gates (orange dashed 0.3, dotted 0.5) and the no-rollback ablation, vs feed error; grey line = assumed 30%.
App: per-arm columns	Records correct in this window and running accuracy, with change since last round.
App: gate panel	During drift: the three traffic checks, trust score vs threshold, reputation, and the traffic-only vs full gate decisions.
App: accuracy chart	Running prequential accuracy per arm; stripes = phases; triangles = accepted updates; hover for values.
App: gate chart	Traffic score (green) and reputation (red) with dashed cut-offs; an update is accepted only if both are above their cut-offs.
App: evidence chart	Attribution ('same direction?'), topology ('mixed traffic?'), temporal ('smooth change?') separately.
Live console	One line per round; yellow VETO, green AUTH, blue DRIFT, purple ROLLBACK, red errors; quarantine verdict with its LLR.
benchmark_<source>.png	F1 of 16 detectors (purple = ours) and F1 vs per-record latency (bubble = model size); up-left is better.
sweep_models_<source>.png	Every defence on 5 base learners: repeat attack (left) and benign drift (right); a good defence is high in both.
data_profile_<source>.png	Feature spread after each processing step, top features, importance by aggregation x time window.
live_experiment_iot23.png	Live run: traffic and drift, rolling accuracy per system, processing time and backlog.
App: Model comparison tab	Metric selector over the benchmark, efficiency scatter, every-defence-on-every-learner bars, poisoned-updates table.
App: Data & features tab	Processing steps, data checks, feature spread, importance, the full feature catalogue, stream layout.
App: Live experiment tab / backend /live	Recorded live run; the /live page starts a new one and draws it as it streams.

15. Testing

143 tests, about one minute (python -m pytest). Tests that need the downloaded datasets skip when they are absent. A mutation check (disabling the veto) makes the headline tests fail, as it should. Adding the quarantine arm is tested to leave the other five arms bit-identical.

File	Covers
test_trust_gate.py	veto, thresholds (inclusive / strict), weight normalisation
test_quarantine.py	SPRT bounds, clamping, stickiness, Monte-Carlo error rates; rollback and replay exactness; commit timing; arm isolation; robustness and lag behaviour
test_pipeline.py	regression against committed synthetic logs, headline result, callbacks, seeds, feature drift, EWC anchoring
test_real_data.py	N-BaIoT shapes and exact reproduction of committed results; IoT-23 parser on hand-written logs; IoT-23 headline and quarantine results
test_reputation.py	noise model: identity, no mutation, bounds, flips, lag
test_fake_nbaiot.py	fake data format, determinism, loader compatibility

File	Covers
test_backend.py	health, SSE streaming, persistence, validation, auth, concurrency limit, quarantine options, log formatting
test_comparison.py	feature catalogue; temporal split and metrics; all 16 benchmark models; every base learner in every arm; confusion counts; Zeek featurization; live pacing and live results; live and ingest endpoints; dashboards render

16. How to run

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pytest

.\.venv\Scripts\python.exe experiments\run_scenarios.py --source real
      [--feature-drift] [--rep-flip 0.3 --rep-sigma 0.05] [--rep-lag 300] [--seed N]
.\.venv\Scripts\python.exe experiments\make_plots.py --tag real
.\.venv\Scripts\python.exe experiments\sweeps.py quarantine --source real

.\.venv\Scripts\python.exe -m uvicorn backend.server:app --port 8765   # console :8765
.\.venv\Scripts\streamlit.exe run app.py                               # app     :8501
.\.venv\Scripts\python.exe experiments\benchmark.py --source real       # model comparison
.\.venv\Scripts\python.exe experiments\sweeps.py models --source real    # defences x learners
.\.venv\Scripts\python.exe experiments\data_profile.py --source real     # data and features
.\.venv\Scripts\python.exe experiments\live_experiment.py --speed 600    # live experiment
.\.venv\Scripts\python.exe experiments\live_feed.py CONN_LOG --follow    # live Zeek sensor
.\.venv\Scripts\python.exe docs\build_report.py                        # this PDF
```

Datasets go in data/nbaiot/ and data/iot23/ (gitignored); download instructions are in README.md and PROJECT_GUIDE.txt.

17. Limitations and future work

- The reputation feed is simulated from ground truth; integrating a real threat-intelligence or C2-cost source is Project-II work.
- The SPRT assumes independent per-record feed errors of at most 30%; above that it fails. Correlated errors (a whole blocklist going stale) are not modelled.
- The quarantine hold was raised from 5 to 20 rounds after seeing the lag sweep; other thresholds are untuned defaults.
- Scenario-2 pacing is constructed by interpolation; every endpoint is a real record, but the schedule is imposed.
- On N-BaIoT, seeds only reshuffle calibration rows, so its confidence intervals reflect calibration-order sensitivity, not dataset variability.
- One device pair per dataset; more devices and CICIoT2023 would strengthen the generality claim.
- The scenario detector is trained on 400 rows; it is accurate on the device it saw (0.995) but raises false alarms on 36% of an unseen device's benign traffic. Larger calibration sets or safe adaptation are needed in deployment.
- The live experiment replays a real capture in real time; it is not packet capture on this machine (that needs Zeek or Npcap). The ingest API accepts a real sensor's conn.log but scores it without the gate (no reputation feed).
- Benchmark latencies are single-threaded Python on one laptop; they compare models with each other, not with production IDS engines.
- Future work: learn the feed's reliability online, model correlated feed errors, adaptive hold length from measured feed latency, and adaptive attackers who target the provisional threshold.

References

- Meidan et al., N-BaIoT: network-based detection of IoT botnet attacks using deep autoencoders, IEEE Pervasive Computing, 2018 (UCI dataset 442).
- Garcia, Parmisano, Erquiaga, IoT-23: a labeled dataset with malicious and benign IoT network traffic, Stratosphere Laboratory, 2020.
- Bifet and Gavalda, Learning from time-changing data with adaptive windowing (ADWIN), SDM 2007.
- Wald, Sequential tests of statistical hypotheses, Ann. Math. Stat., 1945.
- Kirkpatrick et al., Overcoming catastrophic forgetting in neural networks (EWC), PNAS 2017.
- Montiel et al., River: machine learning for streaming data in Python, JMLR 2021.
- Rubinstein et al., ANTIDOTE: understanding and defending against poisoning of anomaly detectors, IMC 2009.
- Kravchik et al., Can't boil this frog, arXiv 2002.02741, 2020.
- Korycki and Krawczyk, Adversarial concept drift detection under poisoning attacks, arXiv 2009.09497, 2020.
- Jordaney et al., Transcend, USENIX Security 2017; Yang et al., CADE, USENIX Security 2021; Han et al., OWAD, NDSS 2023.
- Kumar, Fok, Thing, ExpIDS: a drift-adaptable NIDS with improved explainability, arXiv 2509.20767, 2025.
- Nelson et al., Exploiting machine learning to subvert your spam filter (RONI), LEET 2008; Cao et al., FLTrust, NDSS 2021.
- Crab: certified recovery from poisoning in federated learning, arXiv 2401.08216, 2024; AGAT-FL, PeerJ Computer Science, 2025.
- Zhu et al., A distributed sequential algorithm for collaborative intrusion detection networks, arXiv 1002.3190, 2010.